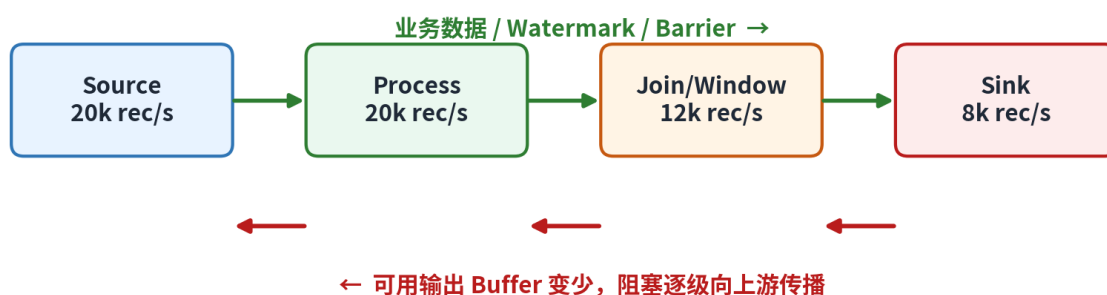


Flink 原理详解：第 7 讲

Backpressure、网络缓冲与流控：下游一慢，为什么全链路都会减速？

本讲目标：不把 Backpressure 只记成“下游慢了”。要理解它如何通过有限网络 Buffer 逐级向上游传播，怎样从 Web UI 找到真正瓶颈，以及为什么它会影响延迟、Watermark、Checkpoint 和 Source Lag。

Backpressure：数据向下游流，压力向上游传



Backpressure 不是“数据倒着流”，而是下游消费不动时，上游被迫减速。

图 1 数据流向下游，Backpressure 沿相反方向向上游传播

1. Backpressure 到底是什么？

Backpressure（反压）是流式系统中的一种流控机制：当下游算子消费数据的速度低于上游产生数据的速度时，系统不会无限制地把数据堆在内存里，而是逐渐让上游减速。

一句话：下游“吃不动”时，上游不能继续无脑“往嘴里塞”，必须被迫降低发送速度。

注意：Backpressure 不是故障本身。它首先是一种保护机制。真正的问题是：为什么某个下游环节长期消费不动？

2. 为什么流处理系统必须有反压？

最简单的数学直觉：生产速率 $\lambda >$ 消费速率 μ ，积压就持续增长



若无限排队：内存最终被吃光；若有限缓冲：缓冲填满后，上游必须减速。

Backpressure 的价值：把“无限积压导致 OOM”变成“受控降速”，让系统保持稳定。

图 2 当生产速率持续高于消费速率时，积压会线性增长

假设 Source 每秒产生 10,000 条记录，下游只能处理 8,000 条。每秒就有 2,000 条新积压。10 秒是 20,000 条，10 分钟就是 1,200,000 条。如果队列无限增长，最终只会得到两个结果：内存耗尽，或者延迟不断扩大到业务不可接受。

所以 Flink 使用有限网络缓冲。当缓冲逐渐填满时，上游会失去可用的输出空间，被迫暂停或降低发送速率。

从系统设计角度看：有限 Buffer + Backpressure 是“用吞吐减速换系统稳定”，避免无限排队。

3. Flink 的网络 Buffer 在这里扮演什么角色？

网络缓冲：Flink 不会每条记录就单独发一个网络包

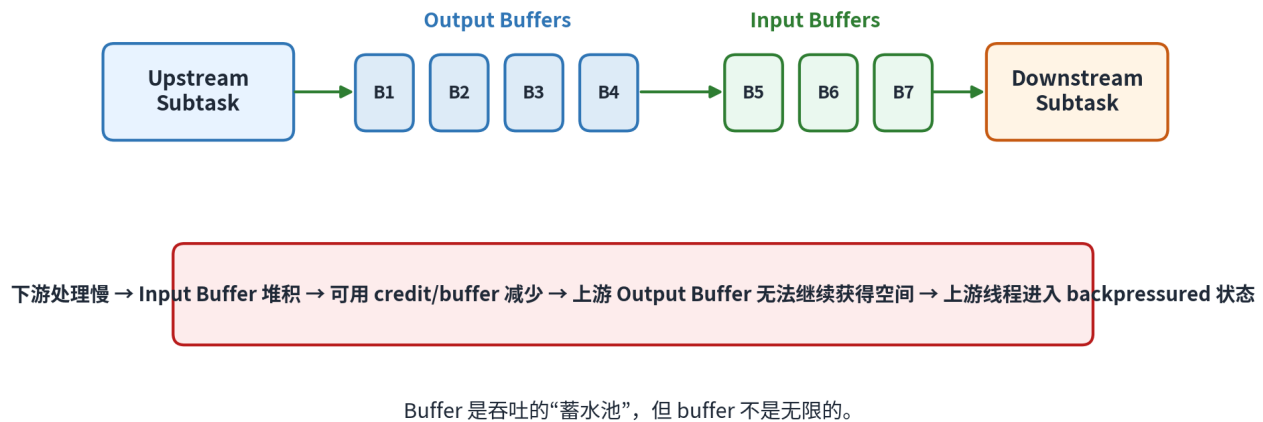


图 3 Record 会进入网络 Buffer；Buffer 也是 Backpressure 传播的关键媒介

Flink 的不同 Subtask 之间通常不是“一条记录一个网络包”。记录会被序列化并装入网络 Buffer，以提高吞吐。上游有输出侧缓冲，下游有输入侧缓冲，这些缓冲中的数据称为 in-flight data（在途数据）。

当下游正常消费时，Buffer 会不断被回收并重新使用；当下游处理速度下降时，输入队列堆积，可用缓冲逐渐减少。最终，上游拿不到新的输出 Buffer，于是进入 backpressured 状态。

官方 Web UI 对 Backpressure 的判断本质上也与“是否还有可用输出 Buffer”有关。

4. 反压为什么会一路传到 Source？

考虑下面这条实时链路：

Kafka Source -> JSON Parse -> keyBy -> Window -> MySQL Sink

如果 MySQL 一秒只能写 5,000 条，但 Window 每秒产生 12,000 条结果：

1. MySQL Sink 来不及消费，Sink 前面的输入 Buffer 开始堆积。
2. Window 无法继续顺畅发送结果，于是 Window 自己被反压。
3. Window 消费输入变慢，Parse / keyBy 上游的 Buffer 也逐渐填满。
4. 反压继续向上游传播，最终 Source 读取 Kafka 的速度也下降。

所以你在 Web UI 里看到 Source 变黑/High Backpressure，并不意味着 Source 自己是根因。它可能只是被最下游的慢 Sink “顶”回来了。

5. Backpressure 会造成什么后果？

影响	表现	底层原因
吞吐	整条流水线吞吐被最慢环节限制	稳定状态下 throughput 受 bottleneck 约束
延迟	事件在 Buffer / Queue 中等待更久	排队时间增加
Source Lag	Kafka 等 Source 的消费位置落后	Source 被迫减速
Watermark	时间进度传播可能变慢	Watermark 与数据同样走数据流
窗口结果	Event-Time Window 可能更	Watermark 到达下游更慢
Checkpoint	start delay / alignment time 可能升高	Barrier 在拥堵链路中传播变慢

Backpressure 为什么会拖慢 Checkpoint？

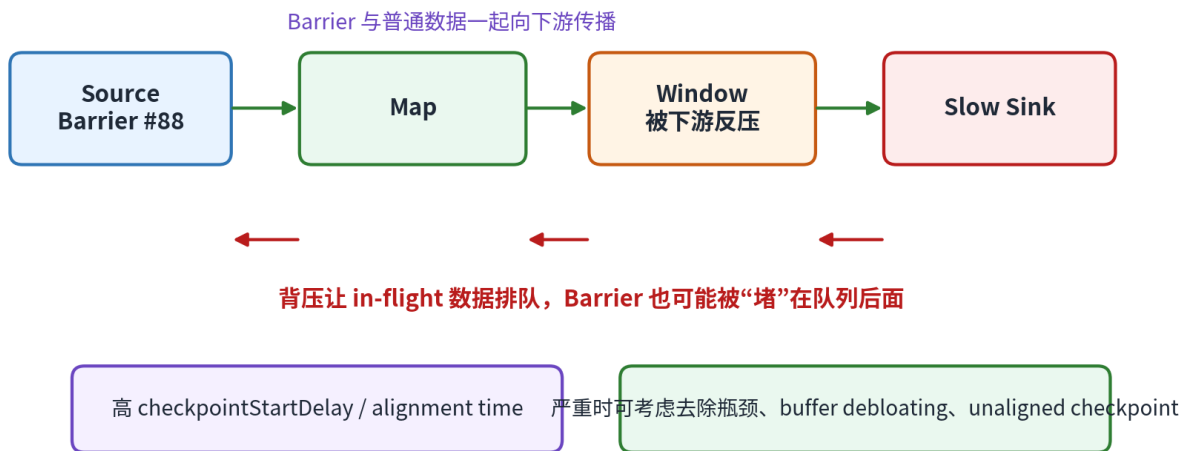


图 4 Backpressure 会影响 Barrier 的传播，因此也会影响 Checkpoint

6. Backpressure 与 Checkpoint 的关系

上一讲已经学过：Checkpoint Barrier 会从 Source 向下游传播。在严重反压时，网络中存在较多排队的 in-flight data，Barrier 也可能需要等待这些数据向前推进。

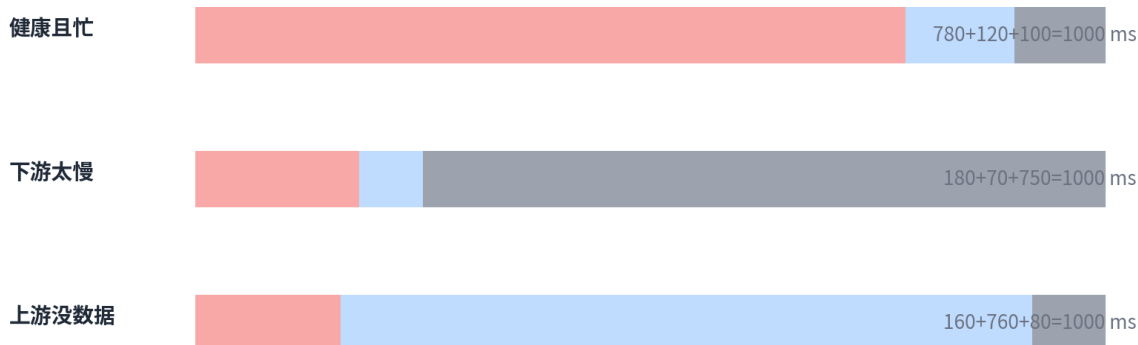
因此你会看到两个很有价值的指标：

- `checkpointStartDelayNanos`：Checkpoint 创建后，Barrier 到达某个 Task 并真正开始 checkpoint 的延迟。高值常提示上游存在 Backpressure；如果只有某个 Task 特别高，也要怀疑数据倾斜。
- `checkpointAlignmentTime`：多输入算子等待同一 Checkpoint 的 Barrier 对齐所耗费的时间。

排查顺序：先解决真正的吞吐瓶颈；若仍有严重 Backpressure，再考虑减少 in-flight data 或使用 Unaligned Checkpoint。

7. Web UI：三个指标怎样看？

Web UI 三个核心时间指标：Busy / Idle / Backpressured



红：busyTimeMsPerSecond 蓝：idleTimeMsPerSecond 灰：backPressuredTimeMsPerSecond
每个 Subtask 每秒大约分成“忙 / 闲 / 被反压”三类时间，合计约 1000 ms。

图 5 每个 Subtask 的 busy / idle / backpressured 时间约合计为 1000 ms/s

Flink Web UI 对每个 Subtask 暴露三个非常实用的时间指标：

指标	通俗含义	高值通常提示
busyTimeMsPerSecond	这一秒里真正忙着执行算子逻辑的时间	CPU/UDF/状态/外部调用等可能成为瓶颈
idleTimeMsPerSecond	这一秒里没有数据可处理、在等待的时间	上游供数不足，或该 Subtask 没吃满
backPressuredTimeMsPerSecond	这一秒里因为下游无法接收而被阻塞的时间	问题通常在更下游

官方文档中，这三类时间在任意时刻加起来大约为 1000 ms。Web UI 还会根据 backpressure 比例给出 OK / LOW / HIGH 等状态。

初学者最重要的判断：高 Backpressure 往往是“症状”；高 Busy 且自己不怎么被反压的下游算子，更接近“根因”。

8. 怎样沿着反压链找到真正瓶颈？

定位反压源：沿着“反压链”往下游追，找到边界

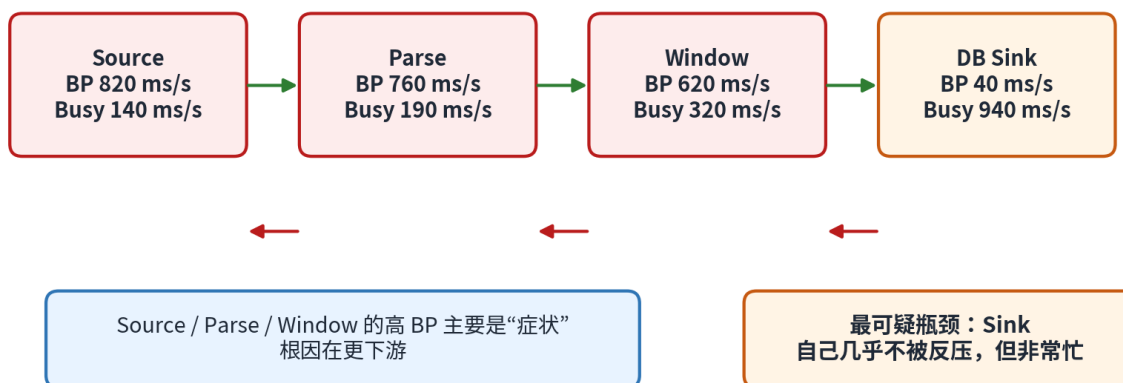


图 6 反压链条上的上游常是受害者；根因通常在反压突然下降而 Busy 很高的位置

5. 打开 Flink Web UI，先看整张 JobGraph 上哪些 Operator 出现高 Backpressure。
6. 从高 Backpressure 的上游算子开始，沿数据方向往下游走。
7. 如果下一个算子也高度 Backpressured，继续往下游找。
8. 找到“上游高度 Backpressured，但这个算子自身 Backpressure 很低、Busy 很高”的边界。
9. 再结合该算子的业务性质判断：CPU、状态、热点 Key、外部 Sink、GC、网络，还是资源不足。

不要看到 Source HIGH 就去“优化 Source”。先问：是谁让 Source 没地方继续输出？

9. 常见根因：不是所有反压都靠“加机器”解决

常见反压根因与第一优先动作

根因	证据	优先动作
Sink 外部系统慢	数据库写入、API、Kafka broker 吞吐不足	批量/异步写；扩容外部系统；提高 Sink 并行度
CPU-heavy Operator	解析、序列化、复杂 UDF、模型推理	优化代码；内置算子；增加并行度/CPU
热点 Key / 倾斜	某个 Subtask 特别忙，其他较闲	重设计 key；拆热点；分片/两阶段聚合
状态/窗口太重	大窗口、频繁触发、大 State	增量聚合；减少状态；TTL；合理窗口
GC / 内存压力	长 GC、对象创建多、RocksDB/Heap 压力	分析 GC；减少对象/状态；调整内存与后端
网络 / Buffer	跨节点带宽不足、in-flight 过多	先证据诊断；buffer debloating；谨慎调网络内存

原则：先找最慢环节，不要一看到 Backpressure 就先“加大 Buffer”。

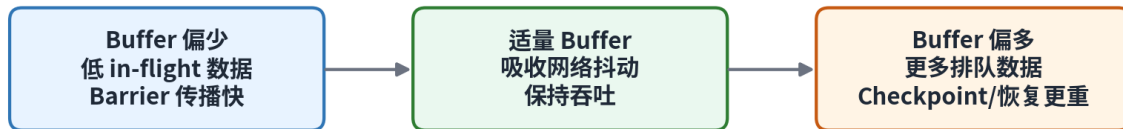
图 7 反压的根因不同，解决方法完全不同

几个很典型的现实场景：

- 慢 Sink：MySQL、REST API、外部文件系统写入速度不足。增加上游并行度反而可能把压力打得更大。
- 热点 Key：总体资源很多，但某一个 Key 全部落到同一个 Subtask，造成单点满载。
- CPU-heavy UDF：复杂 JSON、正则、模型推理、序列化等让算子本身计算不动。
- 大窗口 / 大 State：状态操作、序列化、RocksDB I/O 或定时器数量过多。
- GC / 内存压力：JVM 在长时间回收内存，业务线程实际处理时间下降。
- 网络瓶颈：跨节点吞吐受限，Buffer 长期排队。

10. Buffer 是越大越好吗？

Buffer 的经典权衡：吞吐、延迟、Checkpoint 之间没有免费午餐



Buffer Debloating：根据实际吞吐自动调整 in-flight 数据量，使缓冲数据的目标消费时间维持在合理范围。

它是“缓冲量自适应”，不是解决 CPU、热点 Key、慢数据库等根因的魔法按钮。

图 8 Buffer 能提高吞吐弹性，但过多 in-flight data 会带来新的成本

Buffer 的作用类似公路上的缓冲车道：太少会让短暂抖动立刻传成停顿；但太多又意味着系统里同时“悬着”更多未完成数据。

Buffer 过多可能带来：

- 更多网络内存占用；
- 更长的排队延迟；
- Aligned Checkpoint 中 Barrier 传播更慢；
- Unaligned Checkpoint 中要保存更多 in-flight data，Checkpoint 体积更大；
- 故障恢复时需要恢复更多在途数据。

因此不要把“增加 Buffer”当成反压的默认解法。它经常只是把积压藏得更深、延迟拖得更长。

11. Buffer Debloating：让缓冲量跟着吞吐自适应

Flink 提供 Buffer Debloating 机制，可以根据测得的吞吐，自动调整 in-flight data 的量，让“当前缓冲数据被完全消费掉”大致接近一个目标时间。

```
taskmanager.network.memory.buffer-debloat.enabled: true
taskmanager.network.memory.buffer-debloat.target: 1 s
```

这类机制特别适合解决“Buffer 过多导致 Checkpoint / 恢复负担过重”的问题。但是它不是万能性能开关。

如果真正瓶颈是 MySQL 每秒只能写 5,000 条，Buffer Debloating 不会突然把 MySQL 变成 50,000 条/秒。根因仍然要解决。

12. 一个完整案例：实时订单系统为什么突然越来越慢？

假设链路为：

```
Kafka(30k/s) -> Parse(30k/s) -> keyBy(user) -> FraudCheck(28k/s) -> MySQL Sink(10k/s)
```

运行 20 分钟后出现：Kafka lag 持续上升；Source HIGH Backpressure；FraudCheck HIGH Backpressure；MySQL Sink Busy≈950 ms/s，但 Backpressure 很低。

正确推理过程：

10. Source 高 Backpressure 不是根因，因为它只是读不动 Kafka。
11. FraudCheck 也被下游顶住；说明 FraudCheck 虽然忙，但还不是最末端瓶颈。
12. MySQL Sink 几乎一直 Busy，而且自己几乎不被反压，说明它就是最可疑边界。
13. 先调库连接、批量提交、事务小、减少 Sink 并行度，而不是先给 Source 加并行度。
14. 优化 Sink 后，再重新观察 Backpressure、Kafka lag、吞吐和 Checkpoint。

性能优化必须形成闭环：证据 → 假设 → 最小改动 → 再测。否则“多配点内存、多加几个并行度”只是碰运气。

13. 手工实验：用一个小程序感受积压与反压

```
producer_rate = 10000 # 每秒产生
consumer_rate = 8000  # 每秒消费
queue_capacity = 12000
queue = 0

for sec in range(1, 11):
    produced = producer_rate
    free = queue_capacity - queue
    accepted = min(produced, free)
    blocked = produced - accepted
    queue += accepted

    consumed = min(queue, consumer_rate)
    queue -= consumed

    print(sec, "queue=", queue, "blocked=", blocked)
```

你会看到：最开始 Buffer 可以吸收差额；当队列逼近容量后，producer 开始出现 blocked。这就是 Backpressure 的最小模型。

现实 Flink 比这个复杂得多，但“有限缓冲满了 → 上游无法继续发送 → 压力向上游传播”的底层直觉完全一致。

14. 常见误区

误区	正确理解
Backpressure = 系统挂了	不是。它首先是流控机制；长期高压才提示持续瓶颈
Source HIGH，说明 Source 慢	往往相反：Source 可能被下游慢算子顶住
增加 Buffer 就能解决	只能吸收短时波动，不能提高真正瓶颈的处理能力
增加并行度一定有效	热点 Key、单线程外部系统、数据库瓶颈可能无效
Busy 高 = Backpressure 高	者不同：Busy 表示自己忙，Backpressure 表示被下游阻塞
只看平均吞吐就够了	还要看单个 Subtask、倾斜、延迟、lag、checkpoint
Unaligned Checkpoint 解决反压	它主要改善反压下 checkpoint 行为，不会消除业务瓶颈

15. 练习题

15. 某 Job 中 Source BP=900 ms/s，Map BP=850 ms/s，Sink BP=30 ms/s、Busy=960 ms/s。最可疑瓶颈是谁？为什么？
16. 生产速率 15,000/s，消费速率 12,000/s。如果没有反压，5 分钟理论上会新增多少积压？
17. 为什么把 Buffer 加大 10 倍可能让系统“看起来短期更顺”，但端到端延迟反而更糟？
18. 为什么 Backpressure 会让 Event-Time Window 的结果更晚出来？
19. 严重 Backpressure 下 aligned checkpoint 很慢，你应该先做哪几类检查，而不是直接切成 unaligned？

16. 参考答案

- 第 1 题：Sink。上游都被反压，而 Sink 自身 BP 很低但 Busy 极高，符合瓶颈边界特征。
- 第 2 题： $(15000-12000) \times 300 = 900,000$ 条。
- 第 3 题：更大 Buffer 只是允许更多数据在途排队，真正消费能力没变；排队等待时间和 checkpoint/恢复负担可能更大。
- 第 4 题：Watermark 同样沿数据流向下游传播；链路堵塞时 Watermark 到达窗口算子的时间也会延后。
- 第 5 题：先定位业务瓶颈（Sink/UDF/倾斜/资源/GC/网络），再看 in-flight data、checkpoint start delay/alignment time；之后才考虑 buffer debloating 或 unaligned checkpoint。

17. 本讲最终心智模型

数据向下游流；下游消费不动 → 网络 Buffer 填满 → 可用输出 Buffer 减少 → 上游被阻塞 → Backpressure 逆向传播。真正调优的目标不是“把反压颜色变绿”，而是找到并提高最慢环节的可持续处理能力。

把第 6 讲和第 7 讲连起来，可以得到一条重要逻辑：

慢下游 → Backpressure → in-flight data 增多 → Barrier 传播/对齐变慢
→ Checkpoint duration / start delay 上升
→ 需要“先治瓶颈，再调 checkpoint / buffer”

18. 下一讲预告：Flink 第 8 讲 - Timer、ProcessFunction 与低层流处理控制

下一讲会进入 Flink 中非常有力量的一组机制：Timer、ProcessFunction、KeyedProcessFunction。我们会回答：如果普通 Window 不够用，怎样让每个 Key 自己“定闹钟”，例如“用户 30 分钟没有新行为就触发一次判断”“订单创建 15 分钟仍未支付就报警”。

官方资料核对

本讲关键机制按 Apache Flink stable 文档核对：Monitoring Back Pressure、Metrics、Checkpointing under backpressure、Network Buffer Tuning。重点事实包括：Backpressure 沿数据流反方向传播；Web UI 使用 busy/idle/backpressured 三类时间；无可输出 Buffer 时 Task 被视为 backpressured；严重 Backpressure 会拖慢 Barrier 传播；Buffer Debloating 可自动调整 in-flight 数据量。